UNIVERSIDADE FEDERAL DO PARÁ
INSTITUTO DE CIÊNCIAS EXATAS E NATURAIS
FACULDADE DE COMPUTAÇÃO

NP-Completude e Técnicas de Projeto de Algoritmos

Alessandro Reali Lopes Silva
Felipe Lisboa Brasil
Gian Victor Goncalves Figueiredo
Enya Clara Elizabeth da Silva Araujo
Leonardo Cunha da Rocha

UFPA / ICEN / FACOMP
Campus Universitário do Guamá
Belém-Pará-Brasil
2026

UNIVERSIDADE FEDERAL DO PARÁ
INSTITUTO DE CIÊNCIAS EXATAS E NATURAIS
FACULDADE DE COMPUTAÇÃO

NP-Completude e Técnicas de Projeto de Algoritmos

AUTORES: **Alessandro Reali Lopes Silva - 202304940049**
Felipe Lisboa Brasil - 202404940029
Gian Victor Goncalves Figueiredo - 202404940017
Enya Clara Elizabeth da Silva Araujo - 202404940021
Leonardo Cunha da Rocha - 202304940050

RELATÓRIO APRESENTADO À DISCIPLINA DE ANÁLISE DE ALGORITMOS DO CURSO DE CIÊNCIA DA COMPUTAÇÃO, COMO PARTE DOS REQUISITOS PARA AVALIAÇÃO SEMESTRAL.

ENVIADO EM: 25/02/2026

DOCENTE:

Prof. Nelson Cruz Sampaio Neto
(ICEN/ FACOMP / UFPA)

1 SATISFABILIDADE

Indique se as afirmativas a seguir são verdadeiras ou falsas e justifique. Considere SAT como o problema da satisfabilidade de fórmulas.

a) Se $L_1 \leq_P L_2$ e $L_1 \in P$, então $L_2 \in P$.

RESPOSTA: Falso. Se L1 é um problema que pode ser resolvido em tempo polinomial, não necessariamente vai garantir que a complexidade ou resolubilidade do problema L2 também seja em tempo polinomial.

b) Se $L_1 \leq_P L_2$ e $L_2 \in P$, então $L_1 \in P$.

RESPOSTA: Verdadeiro. Dado que se é possível fazer uma transformação de L1 em L2 capaz de solucionar ambos problemas em tempo polinomial e dado que L2 é um problema de solução em tempo polinomial, logo, é possível concluir que L1 é um problema também de solução em tempo polinomial.

c) Se $L_1 \in NP$, então $SAT \leq_P L_1$.

RESPOSTA: Falso. A afirmação diz que o problema SAT se reduz a qualquer problema L1 da classe NP. Essa é a definição de um problema NP Difícil, porém, nem todos os problemas na classe NP são NP-Difíceis. Por exemplo, se L1 for um problema da classe P, o SAT não pode ser reduzido a ele, a menos que P = NP, o que provaria que o SAT também é um problema.

d) 2-SAT $\leq_P$ 3-SAT.

RESPOSTA: Verdadeiro. A definição de um problema NP - Completo é que ele está contido em NP e todos os problemas da classe NP podem ser reduzidos a ele em tempo polinomial. O problema de Satisfatibilidade Booleana com 3 literais por cláusula, ou 3 - SAT, é um problema da categoria NP - Completo. Como o 2 - SAT é um problema que pertence à classe P e consequentemente a NP também, por definição, ele se reduz ao 3 - SAT.

e) Se $P = NP$, então algoritmos não-deterministas não são mais poderosos que algoritmos tradicionais (deterministas).

RESPOSTA: Verdadeiro. A classe P representa o conjunto de problemas de decisão que podem ser resolvidos em tempo polinomial por uma Máquina de Turing determinista.

A classe NP representa os problemas solúveis em tempo polinomial por uma Máquina de Turing não-determinista. Se P = NP, significa que qualquer problema que possa ser resolvido utilizando um algoritmo não determinismo em tempo polinomial também pode ser resolvido de forma estritamente determinista no mesmo limite de complexidade de tempo polinomial. Portanto, o não-determinismo não ofereceria vantagem de poder computacional para essa classe de tempo.

f) Assumindo que $P \neq NP$, é possível ter uma heurística de custo polinomial para um problema NP-Completo que, para qualquer dado de entrada, calcule uma resposta cujo valor seja exatamente 10% pior que o valor da solução ótima.

RESPOSTA: Falso. Algoritmos de aproximação garantem chegar a "no máximo"uma certa porcentagem do valor ótimo e não exatamente o valor ótimo. Caso existisse uma heurística polinomial que garantisse que seu resultado R fosse exatamente 10% pior que o valor ótimo O, poderíamos simplesmente executar essa heurística em tempo polinomial para encontrar R e depois calcular O = R/1,1. Isso significaria que é possível encontrar a solução ótima exata de um problema NP-Completo em tempo polinomial, o que implicaria que P = NP. Como o enunciado assume que P é diferente de NP, essa heurística não pode existir.

g) Existe um algoritmo de *Branch and Bound* aplicado ao problema da mochila que possui ordem de complexidade $O(n^2)$, em que n corresponde ao número de itens.

RESPOSTA: Falso. O problema da mochila é categorizado como NP - Completo. Algoritmos de branch and bound são métodos de busca exata que precisam explorar todas as opções de soluções possíveis. Para n itens, o número de combinações é n^2. Portanto, a complexidade de tempo de pior caso de um branch and bound para a mochila é exponencial, logo, $O(n^2)$. Caso existisse um algoritmo exato com complexidade de tempo polinomial aplicado para esse problema implicaria em P = NP.

2 Grafos

Dado um grafo $G = (V, A)$, um conjunto independente de vértices é um subconjunto $V' \subseteq V$, tal que todo par de vértices de V' não é adjacente, isto é, se $x, y \in V'$, então a aresta $\{x, y\} \notin A$.

Um conjunto independente maximal[1] é máximo se todos os outros conjuntos independentes têm cardinalidade menor ou igual. Por exemplo, o conjunto $\{1, 3\}$ na figura abaixo não é um conjunto independente maximal porque o vértice 5 pode ser inserido nesse conjunto, enquanto os conjuntos $\{7, 8\}$ e $\{1, 3, 5\}$ são conjuntos independentes maximais, sendo que o conjunto $\{1, 3, 5\}$ além de maximal é máximo.

Esse problema tem várias aplicações práticas:

a) Suponhamos que você queira realizar uma reunião envolvendo o maior número possível de pessoas do seu círculo de amizades que não se conhecem. Dentre as pessoas que poderiam ser convidadas, você traça um grafo contendo uma aresta ligando duas pessoas que se conhecem. O conjunto independente máximo representa o maior conjunto de pessoas que não se conhecem.

   RESPOSTA: Seja $G = (V, A)$ um grafo não direcionado representando uma rede de distribuição de água, onde cada vértice $v \in V$ representa um reservatório ou a fonte F e cada aresta $\{u, v\} \in A$ representa uma conexão direta entre os reservatórios. Dado que cada reservatório é conectado com a fonte por um único caminho, o grafo G é uma árvore enraizada na fonte F. O problema consiste em determinar um subconjunto $V' \subseteq V \setminus \{F\}$, tal que para todo par de vértices de V' não seja adjacente, isto é, se $x, y \in V'$, então $\{x, y\} \notin A$, e que maximize $|V'|$.

b) Considere um grafo cujos vértices representam projetos que podem ser executados em uma unidade de tempo. Todo projeto que utiliza recursos comuns a um outro projeto é interligado por uma aresta. O conjunto independente máximo representa o conjunto maximal de projetos que podem ser executados simultaneamente em um único período de tempo.

   RESPOSTA:

[1] Um conjunto independente é maximal quando não existe nenhum outro conjunto independente que o contenha, isto é, um conjunto que não pode ser aumentado.

```
Algoritmo ConjuntoMaxArvore(G,F)

Entrada: árvore G = (V, E) e a raiz F
Saída: conjunto independente máximo V' sem a raiz F

d é uma matriz |V| x 2
S ← ∅

1.para cada v ∈ V faça
2.      d[v][0] ← 0
3.      d[v][1] ← 0
4.
5.BuscaProfunda(F, null)
6.
7.Reconstruir(F, null, verdadeiro)
8.
9.retorne S

procedimento BuscaProfunda(v, pai)
1.      d[v][0] ← 0
2.      d[v][1] ← 1
3.
4.      para cada u adjacente a v faça
5.              se u ≠ pai então
6.                      BuscaProfunda(u, v)
7.
8.                      d[v][1] ← d[v][1] + d[u][0]
9.                      d[v][0] ← d[v][0] + max(d[u][0], d[u][1])

procedimento Reconstruir(v, pai, paiEscolhido)
1.      escolhido ← falso
2.      se paiEscolhido = falso e d[v][1] >= d[v][0] então
3.              escolhido ← verdadeiro
4.              adicionar v em S
5.
6.      para cada u adjacente a v faça
7.              se u ≠ pai então
8.                      Reconstruir(u, v, escolhido)
```

Figura 1 – Resposta da letra B

c) **Problema do conjunto maximal de reservatórios críticos:** A rede de uma distribuidora de água foi implantada de modo que vários reservatórios estejam conectados via aquedutos a uma fonte. Em outras palavras, alguns reservatórios podem estar indiretamente conectados à fonte por meio de conexões com outros reservatórios. Entretanto, cada reservatório está conectado à fonte por um único caminho. Caso falte água em algum reservatório, ele pode ser atendido por outro diretamente conectado a ele. A distribuidora de água quer identificar o conjunto maximal de reservatórios críticos, isto é, o maior subconjunto I de reservatórios tal que não exista conexão direta entre nenhum par de reservatórios em I na rede de distribuição. O que fazer:

- Modele o problema de determinar o conjunto maximal de reservatórios críticos usando grafos.
- Apresente um algoritmo baseado em programação dinâmica que resolva o problema.
- Qual a ordem de complexidade do algoritmo?
- Apresente um algoritmo guloso para resolver o problema.

RESPOSTA:

A ordem de complexidade do algoritmo *ConjuntoMaxArvore* é $O(V)$, pois o código é dividido em 3 grandes seções: o primeiro loop `para`, que possui complexidade $O(V)$; o procedimento *BuscaProfunda* que, como o problema foi modelado como uma árvore, possui complexidade $O(V)$; e o procedimento *Reconstruir* que, da mesma forma que o *BuscaProfunda*, possui complexidade $O(V)$.

```
Algoritmo ConjuntoMaxGuloso(G,F)

Entrada: árvore G = (V, E) e raiz F
Saída: conjunto independente máximo V' sem a raiz F

w é uma estrutura que armazena o peso de cada aresta
Q é uma lista de prioridades das arestas ponderadas
P ← ∅
S ← ∅

1.Ponderar(F, null)
2.
3.Crie Q
4.
5.enquanto Q não é vazio faça
6.      (v, u) ← ExtractMin(Q)
7.      se u ∉ P então
8.              adicione u em S
9.              adicione v em P
10.
11. retorne S

procedimento Ponderar(v, pai)
1.w(pai, v) ← 1
2.para u adjacente a v faça
3.      se u ≠ pai então
4.              Ponderar(u, v)
5.              w(pai, v) ← w(pai, v) + w(v, u)
```

Figura 2 – Resposta da letra C

3 ALGORITMO DE PRIMM

Implemente em uma linguagem de programação a sua escolha o algoritmo de Prim. Em seguida, trabalhe os seguintes itens:

a) Encontre manualmente uma árvore geradora mínima para o grafo abaixo e use o programa para validar o resultado. Mostre o log da execução.

RESPOSTA:

Desenvolvimento do Algoritmo de Prim

Passo 1:
Escolhendo o vétice 'T' como inicial, temos os seguintes pesos: $Z(9); Y(8); X(4); S(5)$.
Grafo: $\{T : []\}$.

Passo 2:
Escolhendo o vértice 'X' por ser o de menor peso, temos os seguintes pesos: $Z(9); Y(3); S(5)$.
Grafo: $\{T : [(X, 4)], X : [(T, 4)]\}$.

Passo 3:
Escolhendo o vértice 'Y' por ser o de menor peso, temos os seguintes pesos: $Z(9); S(1)$.
Grafo: $\{T : [(X, 4)], X : [(T, 4), (Y, 3)], Y : [(X, 3)]\}$.

Passo 4:
Escolhendo o vértice 'S' por ser o de menor peso, temos os seguintes pesos: $Z(9)$.
Grafo: $\{T : [(X, 4)], X : [(T, 4), (Y, 3)], Y : [(X, 3), (S, 1)], S : [(Y, 1)]\}$.

Passo 5:
Escolhendo o vértice 'Z' por ser o de menor peso e o único restante.
Grafo (AGM): $\{T : [(X, 4)], X : [(T, 4), (Y, 3)], Y : [(X, 3), (S, 1), (Z, 7)], S : [(Y, 1)], Z : [(Y, 7)]\}$.

```
Vértice inicial: T
Estado inicial do heap: [(4, 'T', 'X'), (5, 'T', 'S'), (8, 'T', 'Y'), (9, 'T', 'Z')]
Grafo: {'S': [('T', 5), ('X', 6), ('Y', 1)],
 'T': [('S', 5), ('X', 4), ('Y', 8), ('Z', 9)],
 'X': [('T', 4), ('S', 6), ('Y', 3)],
 'Y': [('T', 8), ('S', 1), ('X', 3), ('Z', 7)],
 'Z': [('T', 9), ('Y', 7)]}

Iteração 1:
Aresta escolhida: (T, X, 4)
Estado atual do heap: [(3, 'X', 'Y'), (5, 'T', 'S'), (8, 'T', 'Y'), (9, 'T', 'Z'), (6, 'X', 'S')]

Iteração 2:
Aresta escolhida: (X, Y, 3)
Estado atual do heap: [(1, 'Y', 'S'),
 (5, 'T', 'S'),
 (7, 'Y', 'Z'),
 (9, 'T', 'Z'),
 (6, 'X', 'S'),
 (8, 'T', 'Y')]

Iteração 3:
Aresta escolhida: (Y, S, 1)
Estado atual do heap: [(5, 'T', 'S'), (6, 'X', 'S'), (7, 'Y', 'Z'), (9, 'T', 'Z'), (8, 'T', 'Y')]

Iteração 4:
Vértice já visitado: (T, S, 5)
Estado atual do heap: [(6, 'X', 'S'), (8, 'T', 'Y'), (7, 'Y', 'Z'), (9, 'T', 'Z')]

Iteração 5:
Vértice já visitado: (X, S, 6)
Estado atual do heap: [(7, 'Y', 'Z'), (8, 'T', 'Y'), (9, 'T', 'Z')]

Iteração 6:
Aresta escolhida: (Y, Z, 7)
Estado atual do heap: [(8, 'T', 'Y'), (9, 'T', 'Z')]

Iteração 7:
Vértice já visitado: (T, Y, 8)
Estado atual do heap: [(9, 'T', 'Z')]

Iteração 8:
Vértice já visitado: (T, Z, 9)
Estado atual do heap: []


=== Resultado da AGM: ===

Número de operações de push no heap: 8
Número de operações de pop no heap: 8
Número de iterações: 8
Árvore Geradora Mínima (AGM) encontrada: {'S': [('Y', 1)],
 'T': [('X', 4)],
 'X': [('T', 4), ('Y', 3)],
 'Y': [('X', 3), ('S', 1), ('Z', 7)],
 'Z': [('Y', 7)]}
```

Figura 3 – Resposta da questão 3 letra A

b) Qual a ordem de complexidade do programa?

RESPOSTA:

A complexidade de tempo do algoritmo de Prim implementado é $O(E \log V)$, limite teórico rigoroso esperado quando se utiliza uma lista de adjacência combinada a um heap binário. Essa eficiência deriva da análise agregada das operações na fila de prioridade ($heapq$). O laço externo $while$ executa a extração da aresta de menor custo ($heappop$) até E vezes no pior cenário de um grafo denso. Como a reestruturação do Min-Heap a cada remoção exige um custo de $O(\log V)$, o esforço computacional agregado exclusivo dessas extrações atinge o teto de $O(E \log V)$. A integridade dessa complexidade é mantida graças à contenção do laço for interno por meio da estrutura de conjunto (set) dos vértices visitados. Essa estrutura atua como um filtro lógico de custo $O(1)$, barrando o processamento de arestas que apontam para vértices já resolvidos e impedindo que o heap seja inundado com ciclos inúteis (Lazy Prim). Consequentemente, o laço interno não multiplica a complexidade do while de forma aninhada; ele garante que, ao longo de toda a execução do programa, no máximo $2E$ vizinhos sejam avaliados. Como cada inserção válida ($heappush$) custa $O(\log V)$, o custo total das inserções soma-se como $O(E \log V)$, consolidando a ordem de grandeza final do algoritmo de forma inquestionável.

c) Caso mais arestas e vértices fossem adicionados ao grafo, como o desempenho do programa seria afetado? Em outras palavras, o algoritmo de Prim é mais sensível ao incremento de arestas ou de vértices? Usando dados reais, ilustre graficamente sua resposta

RESPOSTA: No primeiro cenário analisado, o algoritmo Prim com aumento do número de vértices, o melhor caso foi quando o grafo era uma Árvore Geradora Mínima (AGM) com a quantidade de vértices variando até 200, e a quantidade de arestas cresceu com base na restrição $E = V - 1$.

O tempo de execução total x vértices comparado com o tempo de exeução das operações Heap x número de vértices apresentaram um comportamento semelhante. Logo, é possível concluir que, para o melhor caso, o algoritmo Prim possuiu complexidade $O(V \log V)$ e as arestas seguindo a função $E = V - 1$. É perceptível isso pelo tempo de execução do Heap, pois o custo logarítmico das operações do Heap dita o ritmo de crescimento do gráfico.

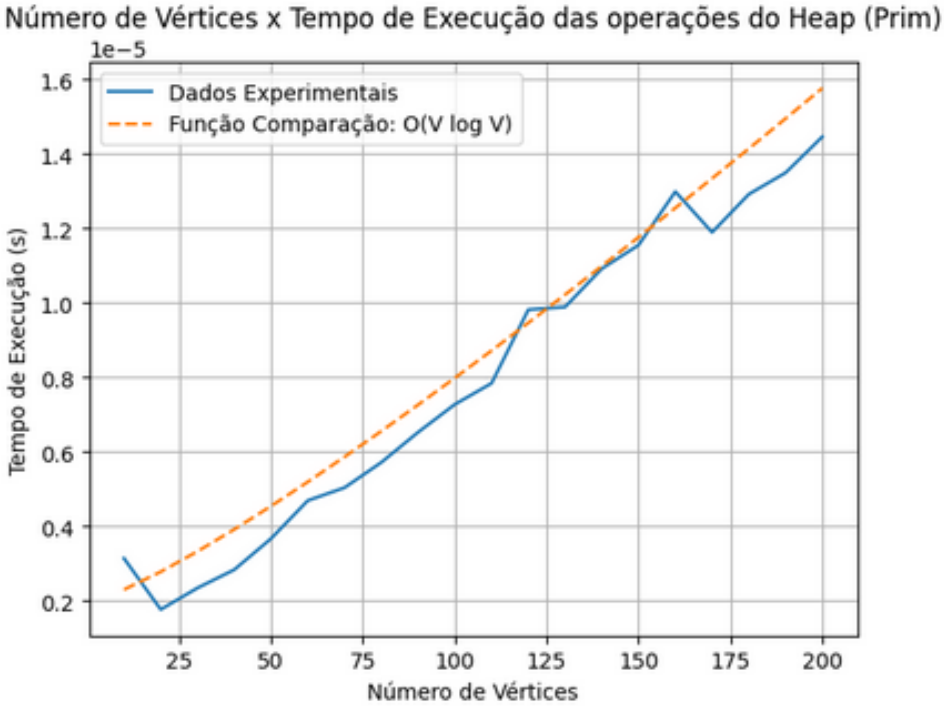

Figura 4 – Gráfico do melhor caso no tempo de operações heap em relação ao número de vértices

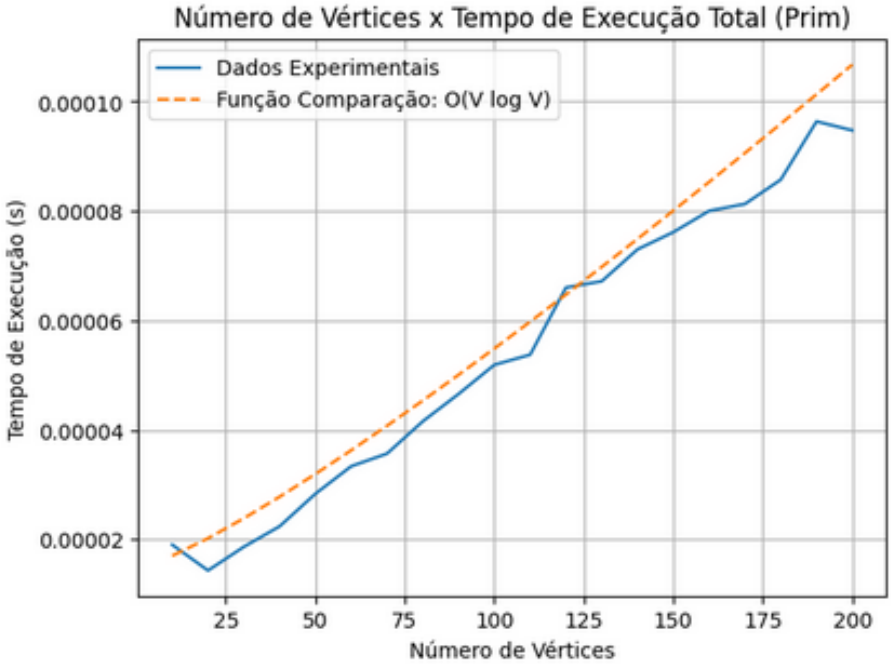

Figura 5 – Gráfico do melhor caso no tempo de execução em relação ao número de vértices

O pior caso foi apresentado quando houveram grafos densos. Quando a densidade foi fixado em 1.0, as arestas são encontradas por $E = V(V-1)$, ainda com os vértices fixados até 200. No pior caso, o grafo completo tem arestas $E = \frac{V(V-1)}{2}$.

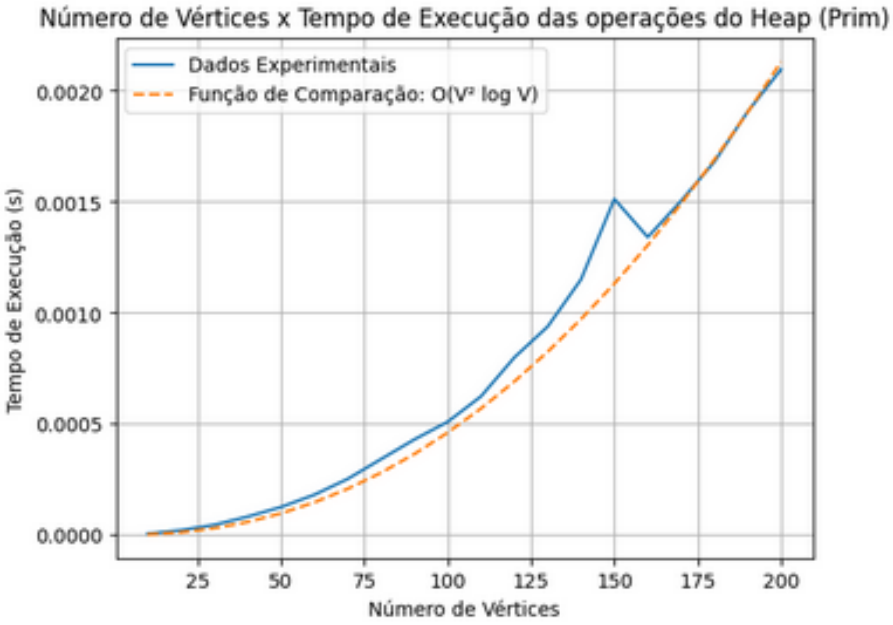

Figura 6 – Gráfico do pior caso no tempo de operações Heap em relação ao número de vértices

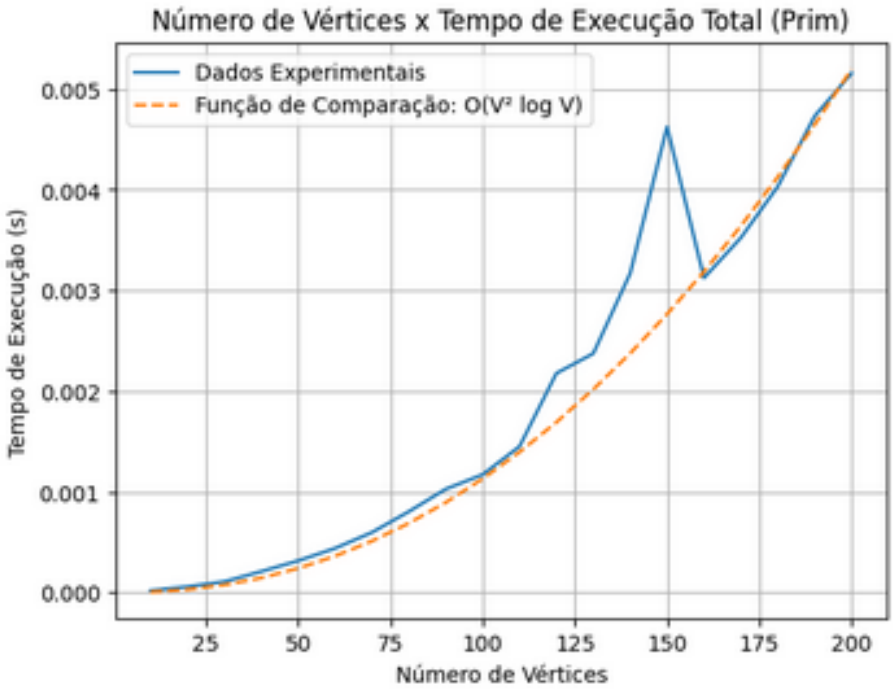

Figura 7 – Gráfico do pior caso no tempo de execuções em relação ao número de vértices

Comparando o tempo de execução das operações Heap com o tempo de execução total, os gráficos apresentaram um comportamento semelhante, exceto que as operações do heap levaram a maior parte absoluta do tempo total de processamento, tornando-se o gargalo do algoritmo.

O que se extrai dos tempos de execução registrados é que o algoritmo de Prim é significativamente mais sensível ao aumento de arestas do que ao de vértices. Visto que ao

manter a densidade, as arestas tiveram complexidade $O(V^2)$, assim como a complexidade do Prim, dado por $O(E \log V)$, transformou-se em $O(V^2 logV)$. Isso é repetido no tempo de execução total e no tempo de execução das operações Heap no pior caso.

Quando o tamanho do problema cresceu pela adição de vértices (de 10 para 200 no cenário de melhor caso), o tempo de execução sofreu um impacto mínimo, subindo de forma muito suave na casa dos microssegundos (de 0,00001s para 0,00009s). O computador gasta pouco tempo "pensando"porque há poucas conexões para avaliar.

Por outro lado, quando o número de vértices foi mantido em 200 e o número de arestas saltou de 199 para 19.900 (pior caso), o tempo total disparou para cerca de 0,005s. O programa ficou mais de 50 vezes mais lento apenas por adicionar caminhos. Isso comprova na prática que o tempo de execução é consumido massivamente pela estrutura do Min-Heap, que precisa processar, inserir e reordenar uma quantidade enorme de conexões repetidas vezes em grafos densos.

O segundo cenário analisado foi o aumento do número de arestas, em que os vértices foram estabelecidos até 200 e a quantidade de arestas variou até 19.900. Ao fixar os vértices, a complexidade do Prim aumenta pela função O(E), e visto que a densidade é proporcional ao número de arestas, pode-se dizer que o aumento da densidade resulta no aumento linear do tempo de execução.

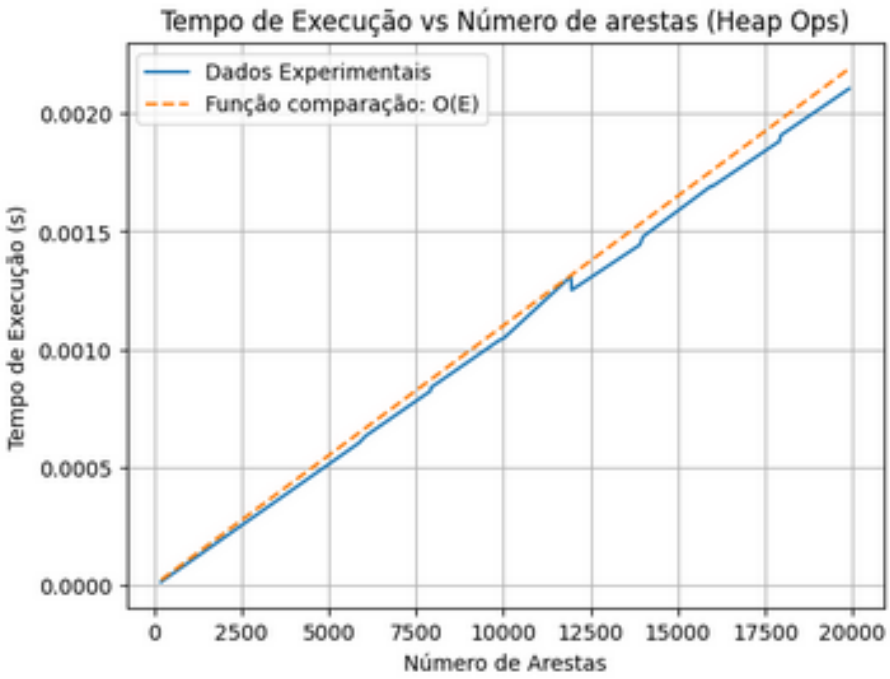

Figura 8 – Gráfico do tempo de operações heap em relação ao número de arestas

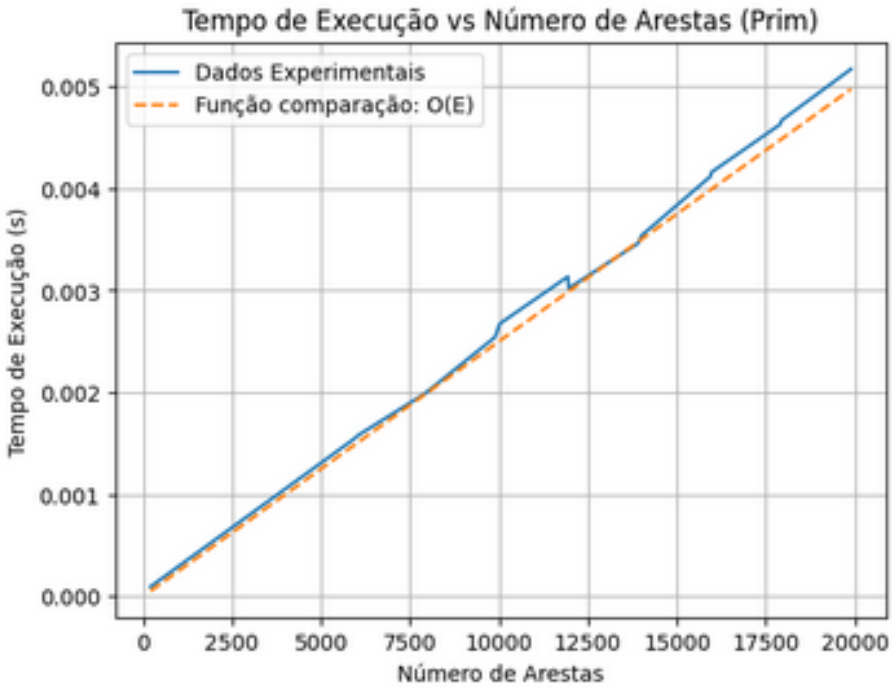

Figura 9 – Gráfico do tempo de execução total em relação ao número de arestas

A comparação do gasto das operações da ordenação com o tempo gasto na totalidade mostra que as operações do Heap equivalem em média a 40% do total do tempo de execução, o que evidencia o Heap influenciar e constituir uma parte significativa do tempo.

Figura 10 – Gráfico referente à participação do Heap nas operações do Prim

As conclusões que são possíveis extrair sobre o algoritmo de Prim é que o melhor caso

obtido é $O(V \log V)$, o caso médio $O(E \log V)$ e o pior caso $O(V^2 \log V)$. Essas complexidades são fortemente influenciadas pela quantidade de vértices.

4 ALGORITMO DE KRUSKAL

Os algoritmos de Prim e Kruskal possuem teoricamente a mesma complexidade no tempo quando Prim usa heap binário e lista de adjacência. Avalie essa afirmativa na prática. Será que esses dois algoritmos realmente se equivalem? Como sugestão refaça a Questão 3.c. para o algoritmo de Kruskal e compare os resultados..

RESPOSTA:

A complexidade do algoritmo Kruskal torna-se $O(V \log V)$ no melhor caso. O número de chamadas de Find cresce linearmente, pois o número de chamadas do Union sempre será o número de arestas da AGM, que é $V - 1$. Nesse caso, haverá uma razão entre o número de chamadas do Union e o número de chamadas do Find, tendendo ao 0.25. Segue tabelas de comparação:

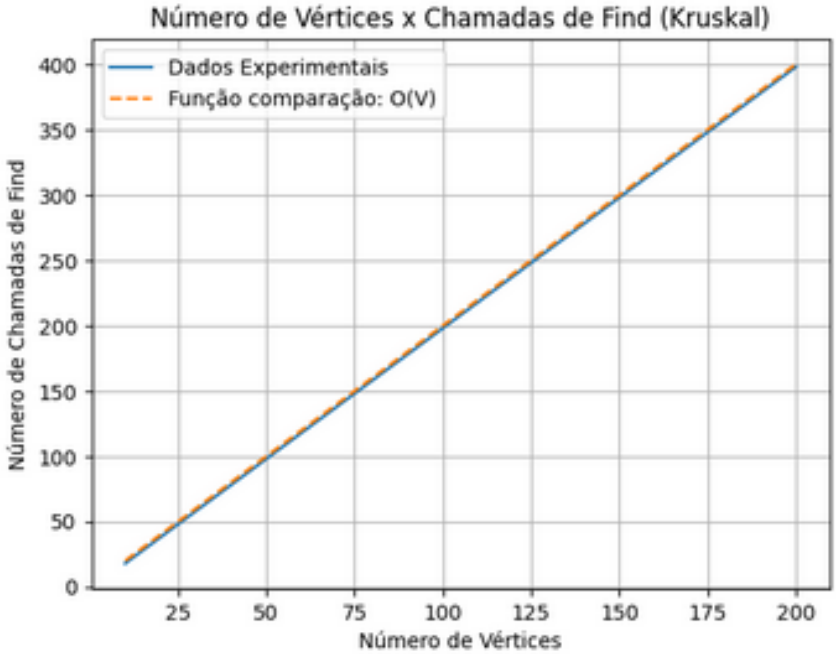

Figura 11 – Gráfico de melhor caso nas chamadas Find

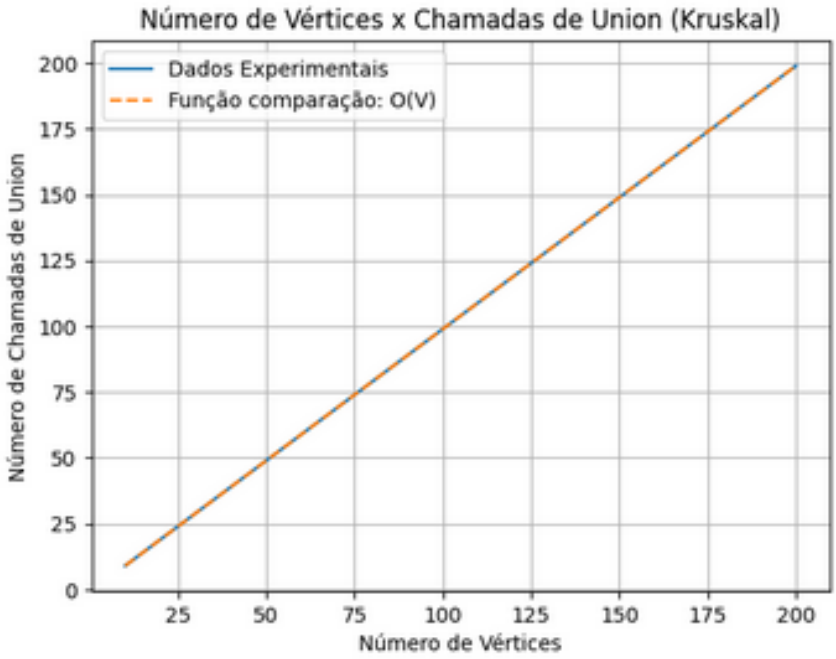

Figura 12 – Gráfico de melhor caso nas chamadas Union

Também é interessante notar que mesmo que o tempo de execução do find e do union são lineares, pois operações envolvendo conjunto são lineares assintoticamente, ou seja tornam-se O(E), que no melhor caso fica O(V). Outro fator importante é que a função de ordenação usada é o TimSort, nativo do python, que em seu melhor caso gera complexidade O(n), no nosso caso, O(V).

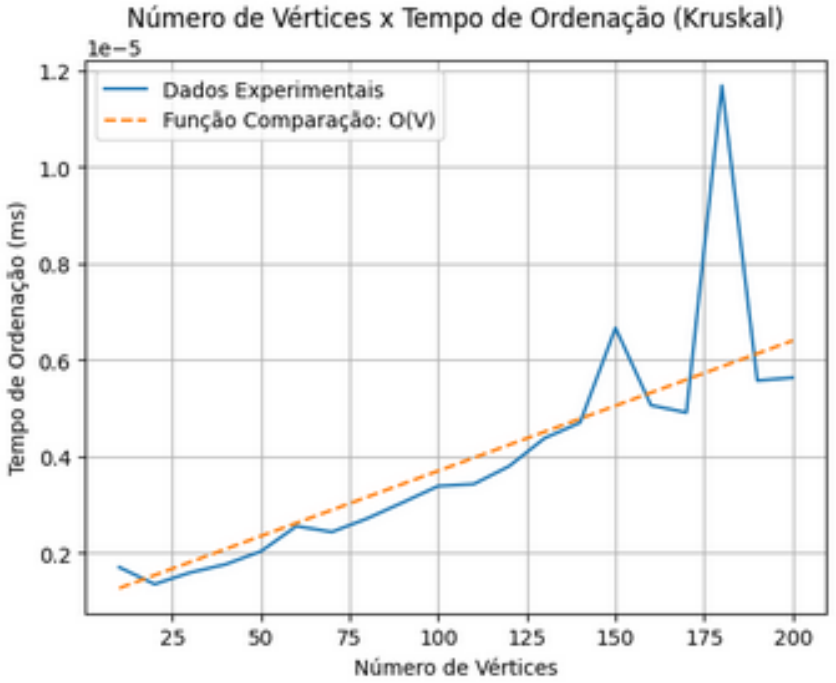

Figura 13 – Gráfico do melhor caso no tempo de ordenação TimSort

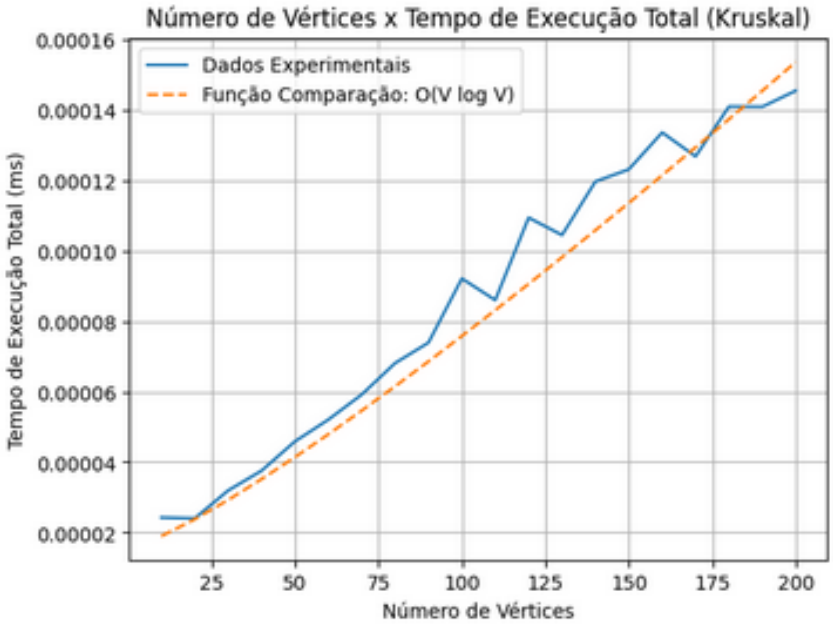

Figura 14 – Gráfico do melhor caso no tempo do tempo de execução total

O pior caso para o algoritmo Kruskal apresentou-se quando houve uma árvore densa. O tempo de ordenação é $O(E \log E)$, que no pior caso se torna $O(V^2 \log V)$. Os tempos de execução do find e do union são lineares em relação à $|E|$, assim tornam-se quadráticos em relação à V. A complexidade do Kruskal é $O(V^2 \log V)$ no pior caso.

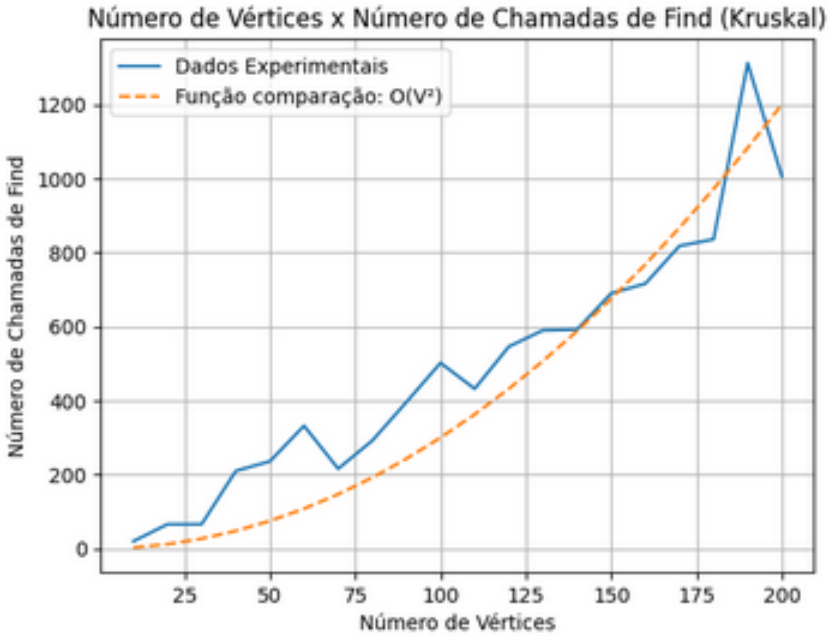

Figura 15 – Gráfico do pior caso no número de chamadas Find nos vértices

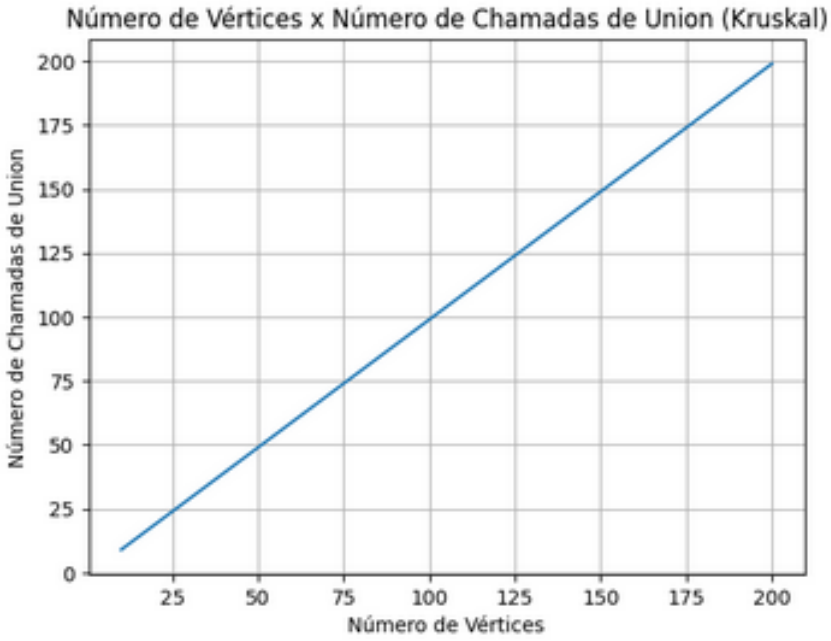

Figura 16 – Gráfico do pior caso no número de chamadas Union nos vértices

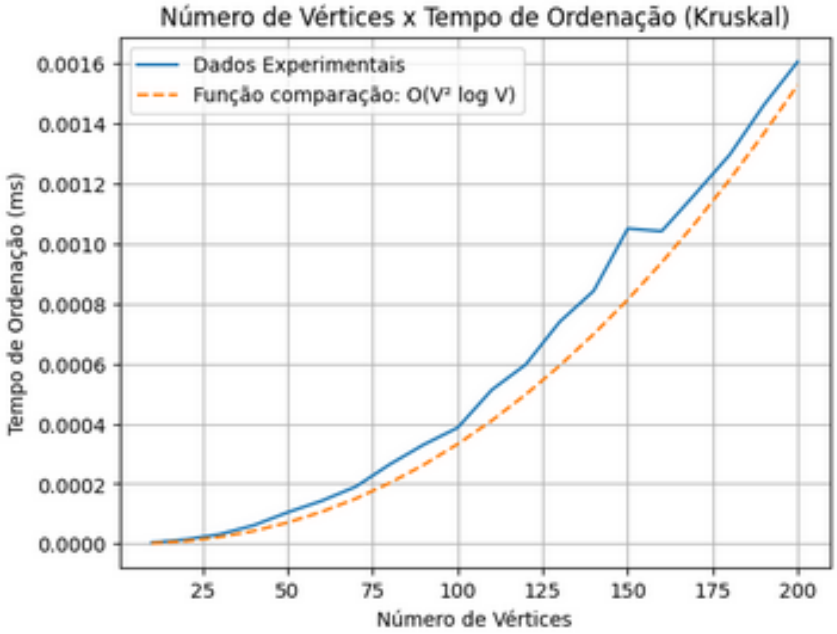

Figura 17 – Gráfico do pior caso no tempo de ordenação total nos vértices

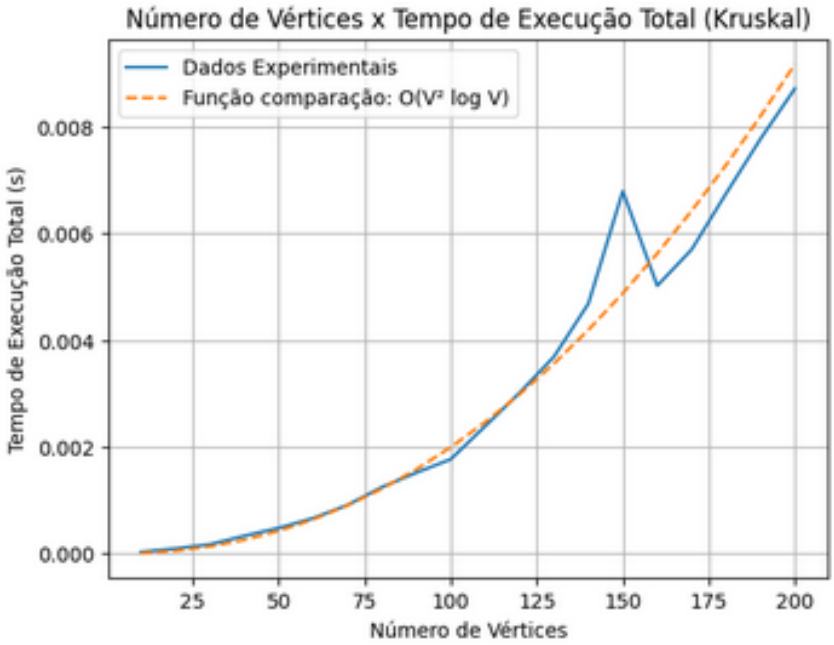

Figura 18 – Gráfico do pior caso no tempo de execução total nos vértices

Para o mesmo número de vértices, o aumento da densidade (número de arestas) corresponde a um aumento linear dos tempos de execução das operações de conjunto enquanto a ordenação cresce em ordem superlinear $(E \log E)$.

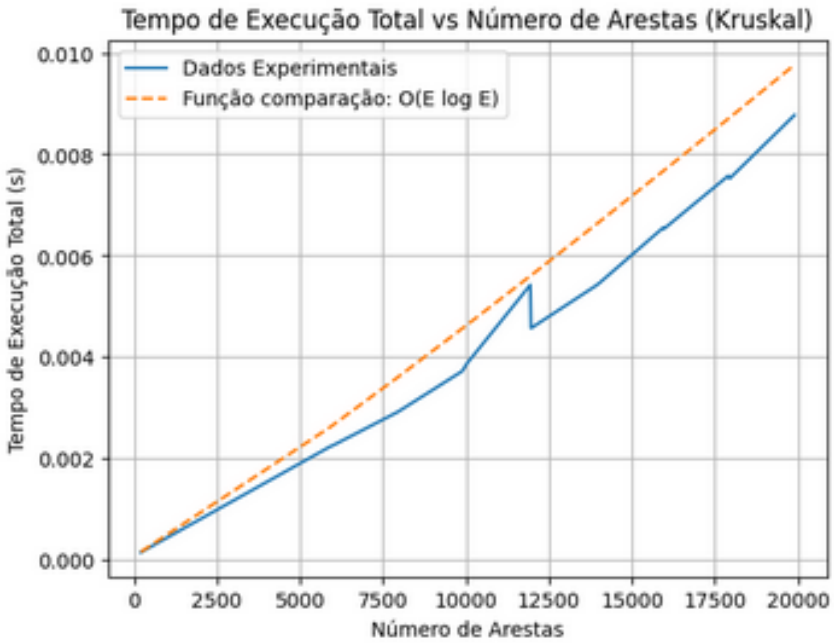

Figura 19 – Gráfico no tempo de execução total nas arestas

Com relação a participação média da ordenação no algoritmo, o gráfico mostra que a ordenação compõe um tempo de 20% do tempo total do Kruskal.

Figura 20 – Gráfico de participação média de ordenação no algoritmo Kruskal

Finalmente, para a comparação entre Prim e Kruskal, foi escolhido os resultados dos vértices que apresentaram melhores melhores casos e os resultados dos vértices que apresentaram piores piores casos entre os dois algoritmos. Nesse sentido, melhor caso para o tempo de execução apresentou-se com o algoritmo Prim, como visto pelo gráfico a seguir.

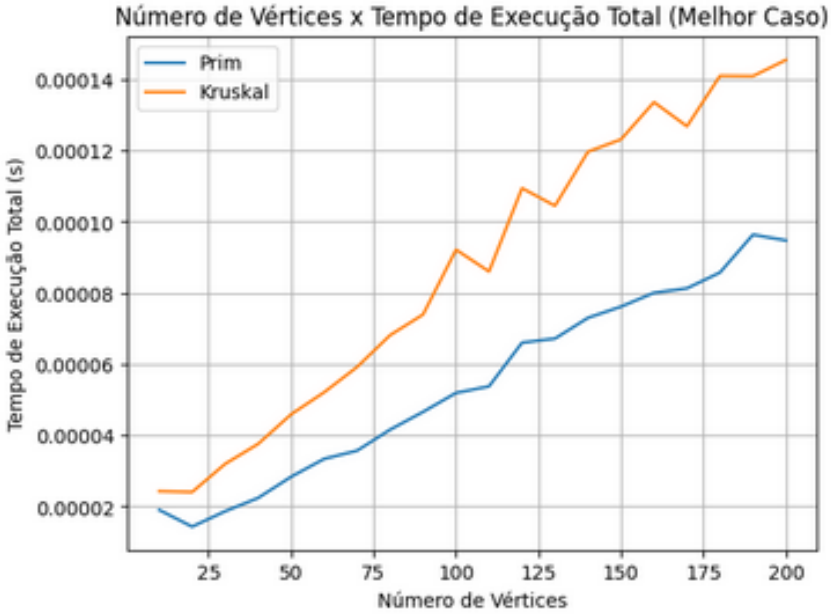

Figura 21 – Gráfico do melhor caso nos tempos de execução total entre Prim e Kruskal

O pior caso para o tempo de execução apresentou-se com o algoritmo Kruskal, como visto pelo gráfico a seguir.

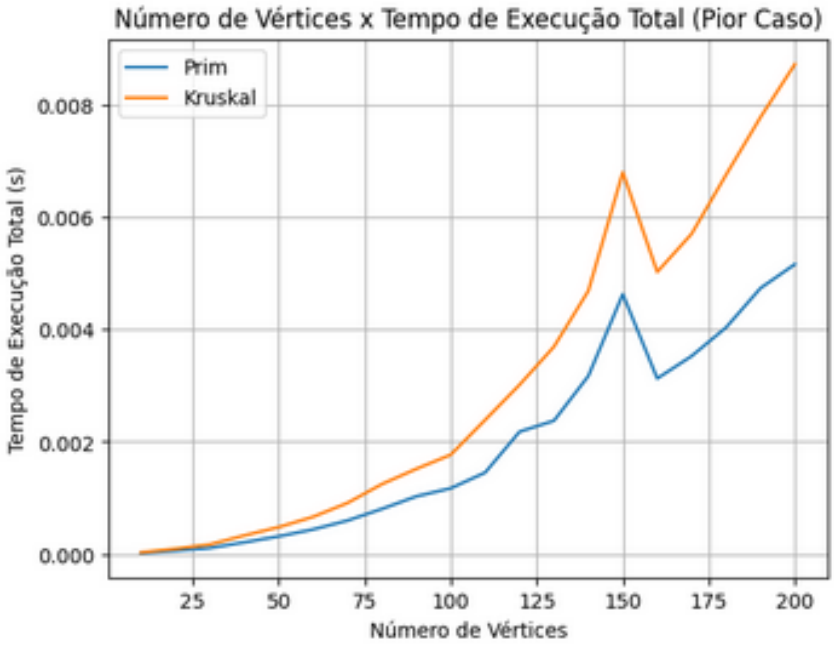

Figura 22 – Gráfico do pior caso no tempo de execução total entre Prim e Kruskal

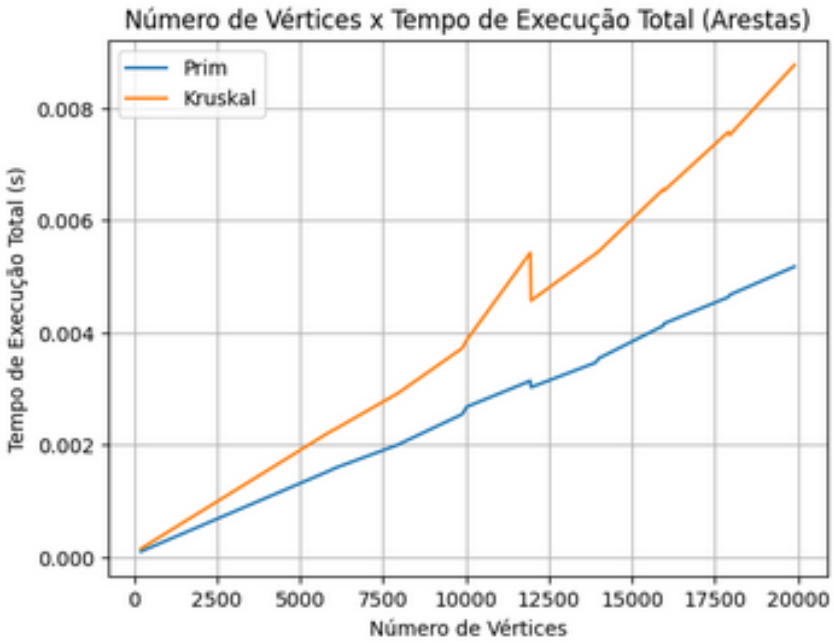

Figura 23 – Gráfico de comparação no tempo de execução total nas arestas

Portanto, a conclusão seguinte com base nos gráficos, ambos os algoritmos apresentam comportamentos semelhantes, o que mais difere é o tempo de execução que cada algoritmo levou nos casos testados. Em cenários de alta densidade, o algoritmo de Prim se mostra uma escolha estruturalmente superior ao algoritmo de Kruskal, que é severamente penalizado pelo custo de ordenação inicial de todas as arestas